# Ders 13 Genişletilmiş Notu: Void İşaretçi, Dosya İşlemlerine Giriş: Stream, Metin ve İkili Dosyalar, fopen Kipleri ve Dönem Projesi

## Ders 13 Genişletilmiş Notu: Void İşaretçi, Dosya İşlemlerine Giriş: Stream, Metin ve İkili Dosyalar, fopen Kipleri ve Dönem Projesi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Void İşaretçi: Genel Amaçlı Bir İşaretçi
- 2. Neden Dosya? Kalıcılık ve İşletim Sistemi
- 3. Stream: Program ile Dosya Arasındaki Köprü
- 4. Metin ve İkili Dosyalar
- 5. FILE Yapısı, Dosya İşaretçisi ve Dosya Konum Göstergesi
- 6. stdio Sabitleri ve fopen Erişim Kipleri
- 7. Okuma ve Yazma Birimleri
- 8. Dosya Nasıl Okunur, Nasıl Yazılır?
- 9. Dosya Biçimleri: Başlık, Veri, Son Ek
- 10. Dosya Açma Kodu
- 11. openFile Fonksiyonu
- 12. Dosya Kopyalama Örneği
- 13. Dönem Projesinin Teknik Yapısı


## 1. Void İşaretçi: Genel Amaçlı Bir İşaretçi

Hoca, önceki derste işlenen fonksiyon işaretçilerinin ardından, yeni konuya geçmeden önce void işaretçiden (void pointer) kısaca söz etti. Void işaretçi jenerik (generic) bir işaretçi türüdür. Normalde `void` denince aklımıza `return` yapılmayan durumlar gelir. Buradaki jenerik sözüyle kastedilen ise başka bir şeydir: Herhangi bir türdeki değişkenin adresini saklayabilen bir işaretçi. `int` işaretçisi, `char` işaretçisi, `float` işaretçisi gibi türlerin hepsini kapsar.

### 1.1 Ne İşe Yarar?

Bazı fonksiyonların jenerik, yani her türle çalışması gerekir. Hocanın örneği, bir dizinin elemanlarını ekrana yazdıran programdır. Dizi `int` dizisi olabilir, karakter dizisi olabilir, kendi tanımladığınız bir yapının dizisi de olabilir. Her biri için ayrı fonksiyon yazarsanız `printIntArray`, `printFloatArray` gibi, başı `print` sonu `array` olan, içi de birbirinden farklı bir sürü isim ortaya çıkar. Oysa tek bir `printArray` ya da `printList` fonksiyonunuz olsa, ona gönderdiğiniz şey `int` dizisi, yapı dizisi, hatta bağlı liste olabilir.

> **Ek bilgi (derste anlatılmadı):** `void*` tek başına verinin türünü ve eleman boyutunu taşımaz. Bu yüzden böyle bir `printArray` fonksiyonunun büyük olasılıkla eleman boyutunu ve elemanı nasıl yazdıracağını ek parametre olarak alması gerekir.

Hoca bu fonksiyonların nasıl yazılacağını bugün anlatmadı. Vakit kalırsa son haftalarda ders notu üzerinden bir örnek göstereceğini, thread konusunda ve çok kanallı programlamada kendi fonksiyonlarımızı yazarken de void işaretçinin işimize yarayacağını söyledi.

### 1.2 Tanım ve Atama

Void işaretçinin tanımı normal bir işaretçi tanımından farklı değildir: `void *ptr;`. Fonksiyon işaretçisi tanımındaki gibi yıldızı parantez içine almaya gerek yoktur. Hoca bir `int i` ve bir `float f` tanımladı. `ptr = &i;` yazdıktan sonra programın ilerleyen bir yerinde `ptr = &f;` de yazabiliriz. Yani aynı işaretçi önce `i`'nin, sonra `f`'nin adresini tutar. Normalde `int` işaretçisini `float` işaretçisine atayamayız. Void işaretçi ise hepsini sağlayabilir.

> **Ders sonrası not:** Uyumsuz işaretçi ataması (`short *s = &i;`, `float *p = &i;`) gcc 15.2 ile `-Wall` altında hata verdi (gcc ile denendi).

### 1.3 malloc ile Bağlantı

Hoca aslında void işaretçiyi `malloc` kullanırken zaten gördüğümüzü hatırlattı. `malloc` ve `calloc`'un dönüş değeri `void*` türündendir. Bu yüzden şöyle yazıyorduk:

```
int *p = (int *)malloc(100 * sizeof(int));
```

Burada yapılan tür dönüşümü (cast), fonksiyonun geriye `void*` döndürmesinden gelir. Böyle bir imkân olmasaydı `int` dizisi için ayrı (örneğin `mallocInt`), `float` dizisi için ayrı bir fonksiyon gerekirdi. Hazır türleri belki hallederdiniz ama kendi tanımladığınız türlerin dizisini bu şekilde oluşturamazdınız.

> **Ders sonrası not:** C'de `void*` dönüşü, cast yazılmadan da `int*` değişkene atanabilir. `int *p = malloc(100 * sizeof(int));` satırı `gcc -Wall` ile uyarısız derlendi (gcc ile denendi). Derste kullanılan cast yazımı da doğrudur.

### 1.4 Dereferans ve Tür Dönüşümü

Void işaretçinin temel kuralı, kullanırken mutlaka asıl türüne çevrilmesidir. Slayttaki ifadeyle *must be casted*.

`ptr`, `i`'nin adresini tutuyorsa `i`'ye `ptr = 10;` diye değer atayamayız, çünkü `ptr` bir işaretçidir ve dereferans edilmesi gerekir. Fakat `*ptr = 10;` de derlenmez. Hoca bunu derleyip gösterdi: Derleyici void işaretçinin dereferans edilemeyeceğini söyler. Çözüm, önce kendi asıl türüne çevirmektir. `*(int *)ptr = 10;` ifadesinde önce tür dönüşümü çalışır, sonra dönüştürülen işaretçi dereferans edilir.

> **Hocanın vurgusu:** Void işaretçi dereferans edilemez. `*ptr` hiçbir şekilde doğrudan kullanılamaz, önce kendi asıl türüne çevrilir.

Derste yazılan kodun derlenen hâli şöyledir:

```
#include <stdio.h>

int main(void) {
    int i = 0;
    float f = 0; /* derste f ilklendirilmedi, çöp değer gösterildi */
    void *ptr;

    ptr = &i;
    *(int *)ptr = 10;
    printf("%d %d\n", i, *(int *)ptr);

    ptr = &f;
    printf("%3.3f %3.3f\n", f, *(float *)ptr);

    return 0;
}
```

Hoca `printf` içinde `i`'nin kendisini ve dereferans edilen değeri yan yana yazdırdı, ikisinin aynı olduğunu gösterdi. `*` işaretini kaldırırsak bu bir adres yazdırmak olur ve bunun için `%p` kullanılır. Ardından `ptr = &f;` ile aynı işlemleri `float` için tekrarladı. Burada önce `f`'nin içindeki çöp değeri görmek istediği için `f`'ye değer vermedi.

> **Ders sonrası not:** Yukarıdaki kod `gcc -Wall` ile uyarısız derlendi. Çıktı `10 10` ve `0.000 0.000` oldu. `f`'nin ilk değeri, uyarısız derlensin ve çıktı belirli olsun diye nota eklendi. `*ptr = 10;` satırı ise gcc'de `invalid use of void expression` hatası verdi (gcc ile denendi).

### 1.5 Dizi Üzerinde Kullanım

Hoca `int a[3] = {1, 2, 3};` dizisini tanımlayıp `ptr = a;` yazdı. `*(int *)ptr` yazdırılınca dizinin ilk elemanı olan 1 görülür. Sonra şu soruyu sordu: Bu bir `int` ya da yapı işaretçisi olsaydı `ptr++` bizi bir sonraki elemana götürürdü. Aynısı burada da geçerli mi?

Çalıştırınca çok farklı bir değer çıktı. Adresleri görmek için `%p` ile `ptr`'nin değerini de yazdırdı. Dizinin ilk elemanı `62fe30` adresinden başlıyordu ve `ptr++` sonrasında adres `62fe31` oldu. Oysa `int` dizisinde adresler dörder dörder artar. Void işaretçi ise yalnız 1 artmıştır. Doğrusu, artışı elle vermektir:

```c
#include <stdio.h>

int main(void) {
    int a[3] = {1, 2, 3};
    void *ptr = a;

    printf("%p %d\n", ptr, *(int *)ptr);

    ptr += sizeof(int);
    printf("%p %d\n", ptr, *(int *)ptr);

    return 0;
}
```

Adım adım: ilk elemanın adresi `0x62fe30` ise, `0x62fe30 + 4 = 0x62fe34` ikinci elemanın adresidir ve okunan değer 2 olur (python ile hesaplandı, ara adım eklendi). Hoca `ptr = ptr + ...` ile `ptr += ...` yazımlarının fark etmediğini de söyledi.

> **Ders sonrası not:** `ptr++` sonrasında okunan "çok farklı değer" dizinin dışından değil, dizinin içinden, baytları kaydırılmış olarak gelir. Küçük uçlu (little-endian) sıralamada dizinin 1-4 numaralı baytları `00 00 00 02` olur ve bu `0x02000000 = 33554432` eder (python ile hesaplandı; aynı değer gcc ile derlenen kodda da çıktı). Hocanın ekranındaki değer altyazıda geçmiyor.

> **Ders sonrası not:** Standart C'de `void*` üzerinde aritmetik büyük olasılıkla tanımlı değildir. GCC bunu 1 baytlık artış olarak yapar. `gcc -Wall` uyarı vermedi, `-Wpedantic` ile `ptr++` ve `ptr += sizeof(int)` satırları uyarı verdi (gcc ile denendi). Taşınabilir yazım `ptr = (char *)ptr + sizeof(int);` biçimindedir; gcc ile denendi ve 2 değerini verdi.

> **Ders sonrası not:** Hoca "`a`'nın kendisi bir pointer" dedi. Dizi adı ifadede ilk elemanın adresine dönüşür, ama dizinin kendisi işaretçi değildir. Bu dizi için `sizeof(a)` 12, `sizeof(ptr)` 8 çıktı (x86-64 üzerinde gcc ile denendi).

### 1.6 void ile void* Farkı

Hoca, void işaretçi kullanan fonksiyonlar olabileceğini de ekledi. Bir fonksiyon `void*` türünde argüman alabilir, `void*` türünde değer döndürebilir, argüman almayabilir de. `malloc`'un tanımındaki gibi, çalışmasını tamamlayınca `void*` döndürmesi bunun bir örneğidir.

> **Hocanın vurgusu:** `void` başka bir şey, `void*` çok çok başka bir şeydir.

## 2. Neden Dosya? Kalıcılık ve İşletim Sistemi

Şimdiye kadar veri girişlerini klavyeden yaptık, çıktıları ekrana yönlendirdik. Bu yöntemin bir dezavantajı vardır: Program kapandıktan sonra bu girdi ve çıktılara dair bilgiler kaybolur. Oysa bilgisayar sistemleri kalıcı sistemlerdir. En basitinden işletim sistemini düşünün, üzerinde bir sürü program çalışır. O hâlde belli verilerin kalıcı bellekte durması gerekir.

Verilerimizin kalıcı olmamasının temel nedeni, programın RAM (ana bellek, main memory) üzerinde çalışmasıdır. RAM uçucu bir bellektir. Girdi ve çıktı işlemlerini HDD veya SSD gibi diskler üzerinden yaparak kalıcılık sağlayacağız. Hocanın tarifiyle kalıcılık şudur: Bilgisayarınızı yakmadığınız, diskinize zarar vermediğiniz sürece, bilgisayar kapansa da elektrik gitse de verilere erişebilmek.

### 2.1 İşletim Sisteminin Rolü

Bu noktada iş, kendi çapımızda yazdığımız programın dışına çıkar. Bilgisayar sisteminde CPU ve RAM vardır, bu ikisi kendi aralarında üç farklı kanaldan haberleşir. Bir de disk vardır. Bu sistemin genel işleyişi işletim sisteminin denetimi altındadır, çünkü orada yalnız bizim programımız yoktur.

> **Ek bilgi (derste anlatılmadı):** Hocanın söylediği üç kanal büyük olasılıkla adres, veri ve kontrol yollarıdır.

Hoca bellekteki duruma benzetti: Dizinin dışına taşınca işletim sistemi segmentation fault hatası verir, çünkü yetkimiz olmayan bir bellek bölgesine erişiriz. Disk üzerinde yetkimiz daha da azdır. Bir yanda ders notlarınız, izlediğiniz filmler, müzikleriniz, özel dosyalarınız, bir yanda işletim sisteminin kendi dosyaları durur. Bu yüzden programlarımız bu işi işletim sisteminin gözetiminde yapmak zorundadır. Sistemin kontrolü ve koordinasyonu işletim sistemine yüklenmiş bir görevdir, bunu ilerideki senelerde ilgili derste göreceksiniz.

İşletim sisteminin dosyalardaki ve cihazlardaki (device) verilere erişim için farklı servisleri vardır. İşletim sistemleri bu konuda birbirinden farklılık gösterir. Bu da her yerde çalışabilecek (portable, jenerik) programlar yazmayı zorlaştırır. Windows'taki dosya biçimi, Linux ve Unix sisteminde farklı olabilir. Aynı sistemde bile Word dosyasının mantığı başka, PDF başka, MP3 başka, PNG başkadır.

### 2.2 C Kütüphanesinin Sağladığı Kolaylık

Programlama dilinin faydası burada ortaya çıkar. Belli çalışma zamanı (runtime) rutinleri kullanarak işletim sisteminin girdi/çıktı işlemlerini kolaylaştıran programlar yazabiliriz. `malloc` nasıl bir fonksiyonsa ve onun sayesinde istediğimiz kadar bellek alabiliyorsak, dosyalar için de benzer fonksiyonlar vardır. Bu fonksiyonlarla dosyayı açacak, içinden veri okuyacak, dosyaya veri yazacak, gerekirse silecek ve kopyalayacağız. Temel işlemler dosyanın oluşturulması, silinmesi, değiştirilmesi ve okunmasıdır. Oluşturmak ve silmek nispeten kolaydır. Okuma ve yazma ise kendi içinde ayrıntılar taşır.

Bu fonksiyonlar genel yapı olarak Unix girdi/çıktı kütüphanelerinden esinlenmiştir. Bir fark vardır: C kütüphanesi daha çok tamponlu girdi/çıktı (buffered I/O) yapar, Unix'in kendi kütüphanesi ise tamponsuz (unbuffered I/O) çalışır. Hoca tamponlamayı bildiğimiz bir sorunla örnekledi. `scanf` ile `%d` okuduktan sonra, başka bir yerde `%c` veya `%s` okumaya çalışınca olmuyordu. "Hocam, klavyeden karakter ya da string okuyamıyorum" sıkıntısının sebebi budur: Girdi tamponlanır, tamponda kalan değer sonraki okumaya yapışır.

> **Ek bilgi (derste anlatılmadı):** Tamponda kalan değer `%d` okumasından sonra kalan Enter karakteridir (`\n`). `5`, Enter ve `x` girdisiyle yapılan denemede `%c` bu karakteri okudu (kod 10), sonraki `%s` ise baştaki boşlukları atlayıp `x`'i okudu (gcc ile denendi).

Unix işletim sistemi ise metin ve ikili dosyaları benzer şekilde görür. Bu iki dosya türüne birazdan geleceğiz.

## 3. Stream: Program ile Dosya Arasındaki Köprü

Çalışan C programımız ana bellektedir, dosyamız ise diskte. Bu ikisi arasında bir akış (stream) kurulur. Hoca buna kanal da dediğini ama tam kanal dememek gerektiğini, bazılarının "ırmak" diye çevirdiğini ve bunun çok da yerinde bir çeviri olmadığını söyledi. İkisi arasında bir bağlantı, bir köprü kurulur ve bu köprü sayesinde veri akar.

Akış mantıksal bir isimdir. Sıralı bir bayt dizisinden, örneğin tek boyutlu bir karakter dizisinden oluşur. Dosyaya yazarken de cihaza yazarken de oluşan şey bir akıştır. `printf` ve `scanf` kullanırken de farkında olmadan arka planda bir akış kullanırız, çünkü ekrana ve klavyeye müdahale etmek doğrudan programımızın yetkisinde değildir. Bu işlemler işletim sisteminin kontrolünde gerçekleşir.

Burada önemli olan, C kütüphanelerinin verinin nerede durduğuna bakmamasıdır. Veri bir teyp sürücüsünde (tape drive) de olabilir, diskte de. `printf`'in alt seviyede nasıl çalıştığını bilmeden ekrana yazdırabiliyoruz. Mühendis olarak bilmek elbette gerekir, ama fonksiyonları kullanmak için şart değildir.

Bir işlem yapabilmek için programımızı bir akış yoluyla bir dosyayla veya bir cihazla ilişkilendirmemiz gerekir.

**Soru:** (Hoca sordu) Tanımda neden "cihaz" (device) kelimesi geçiyor? **Cevap:** Bir öğrenci klavye ya da kamera olabileceğini söyledi. Hoca bunu genişletti: Ağa veri göndermek de bir yazma işlemidir. Ağdan, mikrofondan veya tarayıcıdan veri almak da bir okuma işlemidir.

İşletim sisteminin katmanlı mimarisinde genelleştirilmiş bir `read` ve `write` vardır. Üst katmandaki uygulamalar nihayetinde bunları kullanır. "Şu veriyi şuraya yaz" derler. Bu yazıcıya, ekrana, ağa ya da başka bir çıktı cihazına yazmak olabilir. Altta hepsi için bir `read` ya da `write` bulunur, yukarı çıktıkça ayrıntılar değişir.

### 3.1 Üç Temel Akış

İşletim sistemi her program için otomatik olarak üç akış ayırır:

- **Standart girdi (`stdin`):** Klavye.
- **Standart çıktı (`stdout`):** Ekran, monitör, komut satırına gelen çıktı.
- **Standart hata (`stderr`):** O da standart çıktı gibi ekrana gelir. Aslında çıktıyla ilgilidir, ama olağan dışı hataları normal çıktıdan ayırmak için ayrı bir akış yapılmıştır.

`printf` ve `scanf` bu varsayılan akışları kullanır. Bunların daha genel hâlleri `fprintf` ve `fscanf`'tir. Bunlara nereye yazılacağı ya da nereden okunacağı söylenir. `fprintf(stdout, ...)` normal `printf`'in aynısıdır, `scanf` da aynı şekilde `stdin` kullanır. Hedef başka bir yer de olabilir. Hocanın örneği: Yazıcım belli bir porttaysa, verinin oraya yazılmasını sağlayabilirim.

> **Ders sonrası not:** Hoca önce `fprintf` yerine "`sprintf`" ve `stdout` yerine "`stdin`" dedi, sonra kendisi düzeltti. Dosya açma kodunu yazarken de bir kez daha "`stdin` yazarsam `printf` gibi olur" dedi. Doğrusu `stdout`'tur.

## 4. Metin ve İkili Dosyalar

Hoca iki tür dosyayı ekranda açarak gösterdi. Önce açık bir kaynak dosya gösterdi: İçindeki her şey anlaşılır, "şu karakteri bilmiyorum" diyeceğiniz bir şey yoktur. Buna metin (text) dosyası denir.

Sonra aynı ekranın görüntüsünü alıp hem PNG hem JPEG olarak kaydetti ve bu dosyaları Notepad++ ile açtı. Az önce derlenen programın EXE dosyasını da açtı. EXE içinde bazı parçalar okunabilse de geri kalanı anlamsızdır. PNG ve JPEG dosyalarından da bir bilgi çıkaramayız. Üstelik görüntü iki dosyada da aynı olduğu hâlde içerikleri birbirinden farklıdır, çünkü biçimleri farklıdır. Editörde gördüğümüz bu karakterlerin bile her birinin 0 ve 1 karşılığı olduğunu düşünürsek, içerik daha da anlaşılmaz olur.

En genel hâliyle dosyaları metin ve ikili (binary) olarak iki gruba ayırırız.

### 4.1 Metin Dosyası

Metin dosyası, satırlardan oluşan bir metin akışı içerir. Satırlar Enter yani `\n` karakteriyle biter. Avantajı, en basit metin editöründe bile bütün içeriği insanın okuyup anlayabileceği (human readable) biçimde görebilmektir. C dosyasını açarsınız, kaynak kodu okursunuz. Aldığınız notları açarsınız, okursunuz.

Fakat işletim sistemi dosyaları ve bazı program dosyaları, hocanın deyimiyle şiir yazar gibi satır satır yazılacak dosyalar değildir. İçlerinde farklı veriler vardır ve hepsinin açıktan okunabilir olması gerekmez, hatta yerine göre zararlıdır. Örneğin işletim sistemi için bir bellek yönetimi servisi yazdınız. Derlenmiş hâlinin doğrudan görünmesi, programınızın çalınması ve intihal edilmesi anlamına gelebilir. Dosyaları açtığınızda bazen emoji gibi garip karakterler görmenizin nedeni de verinin ikili yazılmış olmasıdır.

### 4.2 CSV ve Benzeri Biçimler

En yaygın metin biçimi `.txt` dosyalarıdır. Bir de CSV (comma separated values) vardır. Excel'deki sütunlar gibi düşünün, ama sütunlar virgülle ayrılır:

```
id,ad,soyad
1,<ad>,<soyad>
```

Bu biçimde şunu kesin olarak biliriz: Virgül görene kadar okuduğumuz ilk şey `id`, ikinci virgüle kadar okuduğumuz şey `ad`, sonrası `soyad`'dır. Her satır bir kayıttır (record). Enter tuşu aslında satır sonuna `\n` koymak demektir ve kayıtlar böyle alt alta devam eder.

TSV (tab separated values) biçiminde ayraç, sekme (Tab) karakteridir. İsterseniz kendi ayracınızı da belirleyebilirsiniz. Hocanın örneği, alanların arasına iki yıldız koymaktı: `id**ad**soyad`. Bu da sizin dosya biçiminiz olur.

Metin dosyasını okuyan fonksiyonlar veriyi belli bir biçime ve sınıra göre okur: "Virgül görene kadar oku" ya da "10 karakter oku" gibi.

### 4.3 Metin Biçiminin Dezavantajları

İlk bakışta kolay görünse de metin biçimi her zaman iyi değildir. Hoca iki sorun gösterdi:

- **Boyut:** Her karakter, boşluklar dahil, bir bayt yer tutar. Hoca ekrandaki satırın kaç karakter tuttuğunu gösterdi. Bu kadar baytı çok sayıda kayıt için düşünürseniz dosya çok büyür, aktarması, işlemesi, okuması, yazması ayrı dert olur.
- **Ayraç karışması:** Veride virgül geçmesi gerekirse, verideki virgül ile ayraç olan virgül birbirine karışır.

### 4.4 İkili Dosya

İkili biçimde veriler doğrudan yorumlanamayacak şekilde, bir kodlamayla saklanır. Hocanın anlatımıyla, örnek bir addaki her harfin 8 bitlik bir 0-1 karşılığı vardır. Bu bitleri 16'şar 16'şar gruplayıp Unicode gibi bir sisteme göre karşılık gelen karakteri yazabilir, bu şekilde kendi içinde bir sıkıştırma da yapabilirsiniz.

> **Ders sonrası not:** Unicode büyük olasılıkla bir sıkıştırma yöntemi değil, bir karakter kodlama standardıdır. Hocanın anlatımını "veri, editörde okunamayacak biçimde kodlanır" sezgisi olarak almak gerekir. Derste bu kodlama için kod yazılmadı.

## 5. FILE Yapısı, Dosya İşaretçisi ve Dosya Konum Göstergesi

Metin dosyası da ikili dosya da olsa, her girdi/çıktı işleminde bir akışı bir dosyayla ilişkilendirmemiz gerekir. Bunun için `FILE` yapısını kullanırız. Bu yapı `stdio.h` içindedir. Genellikle bir dosya işaretçisi oluşturup (`FILE *fp;`) dosyayı onunla gösteririz. Hoca, yapıların tanımında büyük harf kullanıldığını hatırlatıp `FILE`'ın da bir yapı türü olduğu için böyle yazıldığını söyledi. Bu yapının içinde dosyaya dair bazı üst veriler (metadata) saklanır.

Dosyayı açarken iki şey söylememiz gerekir. Birincisi dosyanın adıdır: Hangi dosya üzerinde işlem yapacağız? İkincisi erişim kipidir (access mode): Dosyayı ne için açıyoruz, okuma mı yazma mı, metin mi ikili mi?

### 5.1 İki Kavram: File Pointer ve File Position Indicator

Hocaya göre sık karıştırılan iki kavram vardır. Bunu şöyle bir şemayla anlattı:

```
my_program  <--- FILE* fp (akış, iki yönlü olabilir) --->  veriler.txt
                                                           ^
                                                           dosya konum göstergesi
```

- **Dosya işaretçisi (file pointer):** `FILE` yapısı içinde tutulan bilgi üzerinden bize dosyayı işaret eder. Programdan dosyaya erişim bunun üzerinden olur. Erişim iki yönlü olabilir, hem okuma hem yazma yapılabilir.
- **Dosya konum göstergesi (file position indicator):** Dosyanın içinde şu an nerede olduğumuzu gösterir. Hocanın benzetmesi komut satırında yanıp sönen imleçtir. Bir bayt konumudur.

Dosyayı açınca konum göstergesi dosyanın ilk baytına yerleşir. Bir bayt oku dediğinizde o baytı okur ve hemen bir sonrakine geçer. Tekrar oku derseniz yine ilerler. Yani dosyanın başından başlar, kaç bayt okutursanız o kadar ilerler. Yazmada da aynıdır. Hoca bunu şöyle örnekledi: Başlangıçta gösterge bir konumdadır. Oraya `A` yazarsanız bir sonraki konuma geçer. Sonra okuma yaparsanız oradaki `B`'yi okur ve yine ilerler.

**Soru:** Bu sürekli ileriye doğru mu gider? **Cevap:** Evet, bir okuma yaptığınızda bulunduğu yerde kalmaz, okuduğunuz bayt kadar ileri gider. Ama "bulunduğun yerden şu kadar geri git" demenin yolları elbette vardır.

Göstergeyi işletim sistemi otomatik günceller. Doğrudan erişemesek de hazır fonksiyonlarla değerini öğrenebilir ve değiştirebiliriz. "4 karakter oku, sonra 2 karakter geri gel" ya da "başa gel" diyebiliriz.

> **Hocanın vurgusu:** Bu iki kavramı iyi bilelim. Dosya işaretçisi dosyayı gösterir, dosya konum göstergesi dosyanın içindeki yeri gösterir.

## 6. stdio Sabitleri ve fopen Erişim Kipleri

`stdio.h` içinde bazı makro sabitleri de vardır. Standart girdi, standart çıktı ve standart hatanın sayısal karşılıkları sırasıyla 0, 1 ve 2'dir. `EOF` (end of file), dosyanın sonuna gelip gelmediğimizle ilgilidir. `NULL` boş işaretçi değerini temsil eder.

> **Ders sonrası not:** 0, 1 ve 2 işletim sistemi düzeyindeki dosya tanımlayıcılarıdır. C'deki `stdin`, `stdout` ve `stderr` ise `FILE*` türündedir. `fileno(stdin)`, `fileno(stdout)` ve `fileno(stderr)` sırasıyla 0, 1 ve 2, `EOF` ise $-1$ çıktı (gcc ile denendi).

Dosyayla çalışmanın ilk adımı onu açmaktır. Bunun için `fopen` kullanılır ve iki argüman alır: dosya adı ve erişim kipi.

```c
FILE *fp = fopen("deneme.txt", "r");
```

Erişim kipleri bir veya iki karakterle yazılır:

| Kip | Anlamı | Dosya yoksa | Dosya varsa |
|---|---|---|---|
| `r` | Metin dosyasını okumak için açar. Okuma baştan başlar. | Açılamaz | İçerik korunur |
| `w` | Yazmak için metin dosyası oluşturur. | Oluşturulur | İçerik silinir |
| `a` | Ekleme (append) kipi. Yazma dosyanın sonuna yapılır. | Oluşturulur | İçerik korunur |

| Kip | Anlamı | Dosya yoksa | Dosya varsa |
|---|---|---|---|
| `r+` | Okuma ve yazma için açar. Gösterge baştadır. | Açılamaz | İçerik korunur |
| `w+` | Okuma ve yazma için yeni dosya oluşturur. | Oluşturulur | İçerik silinir |
| `a+` | Her yerden okunur, yalnız sona yazılır. | Oluşturulur | İçerik korunur |

Hoca `w` kipi için truncate kelimesini açıkladı: Dosyanın içinde ne varsa tamamen silinir, içerik boşaltılır ve konum göstergesi dosyanın başına alınır. Yerine göre kullanışlıdır, ama dikkatli kullanılmalıdır. `r` kipinde böyle bir durum yoktur.

Var olan dosyayı bozmadan yazmak için `a` kipi kullanılır. Gösterge otomatik olarak dosyanın sonuna gider ve mevcut veriler korunur. Konum göstergesini açıkça taşısanız bile yazma işlemi mutlaka dosyanın sonundan devam eder.

Artı işareti (+), okumaya yazmayı veya yazmaya okumayı ekler. `w+` kipi, `w`'nin üzerine okumayı ekler ve o da dosya varsa içini temizler.

İkili kipe geçmek için aynı kiplerin sonuna `b` eklenir: `rb`, `wb`, `ab`, `r+b`, `w+b`. Var olan bir dosyayı ikili kipte okumak için `rb` yazılır. Hocanın özeti şudur: Açmadan önce dosyanın var olması gereken kipler `r` ve `r+`'tır. Eski verinin tamamen silindiği kipler `w` ve `w+`'tır. Biraz ezber gibi görünür, ama kullandıkça oturur.

> **Hocanın vurgusu:** `w` ve `w+` kipleri var olan dosyanın bütün içeriğini siler. Bu kipler dikkatli kullanılmalıdır.

> **Ders sonrası not:** Derste `a` kipi için "var olan dosyayı açar" dendi. Dosya yoksa da oluşturur. Tablodaki davranışlar gcc ile denendi: `r` ve `r+` yok olan dosyada `NULL` döndürdü, `a` ve `a+` dosyayı oluşturdu, `w` ve `w+` içeriği sildi. `a` ve `a+` kipinde `fseek` ile başa gidilip yazılınca da yazı sona eklendi. `a+` kipinde `fseek` ile istenen konumdan okuma yapıldı.

`fopen` geriye bir dosya işaretçisi döndürür. Programın geri kalanında dosyayla bu işaretçi üzerinden çalışırız. Hata olursa `NULL` döner. Hata yoksa dönen değer sıfırdan farklı bir dosya işaretçisidir.

## 7. Okuma ve Yazma Birimleri

Dosyayı açtıktan sonra en çok yapacağımız şey okumak ve yazmaktır. Hoca iki dosya düşünmemizi istedi. Metin dosyasında `a`, `b`, `3` gibi karakterler vardır ve her biri bir bayttır. İkili dosyanın içinde ise 0 ve 1'ler olduğunu düşünürüz. Editörde gördüğümüz içerik de aslında bu bitlerin başka bir gösterimidir.

Metin dosyasından okunabilecek en küçük birim bir bayttır, yani bir karakterdir. Biraz daha büyüğü bir bloktur: bir string, bir sayı. Daha büyüğü bir satırdır. Örneğin `ad,soyad,no\n` biçimindeki bir kayıt bir satırdır, içindeki `ad` bir bloktur, onun içindeki bir harf de bir bayttır. Bu birimler için fonksiyon çiftleri vardır:

- **Karakter (bayt):** `getc` / `putc`. Karakter oku, karakter yaz.
- **Blok:** `fread` / `fwrite`.
- **Satır (string):** `fgets` / `fputs`.
- **Biçimli:** `fprintf` / `fscanf`.

Hoca ilk altısının üzerine `fprintf` ve `fscanf`'i ekleyerek toplam 8 okuma yazma fonksiyonu saydı. İkili dosyalarda da benzer şekilde "bir bit, bir bit dizisi (blok) ya da bütün veri" okunabileceğini söyledi.

> **Ders sonrası not:** Hoca ikili dosyadan "bit okumak"tan söz etti. C'nin dosya fonksiyonlarında büyük olasılıkla en küçük birim bayttır. Tek tek bitlere ulaşmak için bayt okunup bit işlemleri kullanılır.

### 7.1 Satırı Karakter Karakter Okumak

**Soru:** (Hoca sordu) Bir satır okuyacaksınız. Satırı karakter karakter okumak hız açısından mantıklı mı? **Cevap:** Bir öğrenci "olmaz herhalde" dedi. Hoca nedenini şöyle açıkladı: Dosya diskte durur. Satırınızda 20 karakter olsun. Bir seferde bir satır okuyabilen fonksiyon varken karakter karakter okursanız diske 20 kez gidip gelirsiniz. Bu korkunç bir gecikmedir.

Hoca hız farkını kaba rakamlarla anlattı. RAM 7000 küsur MHz seviyesinde çalışır. Kolaylık olsun diye 10 MHz diyelim, bu da saniyede 10 milyon işlem demektir. Sabit disk ise dönerek çalışır, devir sayısını frekansa çevirirseniz kHz seviyesine iner. Hoca arada bin kat fark olduğunu söyledi. Bu yüzden hedefiniz bir bloksa blok fonksiyonlarını, bir satırsa satır fonksiyonunu kullanmak en mantıklısıdır. Bir karakter gerekiyorsa karakter okunur. Ama bir satır için karakterleri tek tek okumak, hocanın deyimiyle saçmadır.

> **Hocanın vurgusu:** Bir satır okumak için karakter karakter okumayın. Disk, RAM'den çok daha yavaştır.
>
> **Ders sonrası not:** Rakamlar yalnız büyüklük farkını anlatan kaba değerlerdir. 10 MHz ile 1 kHz arasındaki oran 10000'dir, MHz ile kHz öneklerinin oranı ise 1000'dir (python ile hesaplandı). Gerçek erişim süreleri büyük olasılıkla buradan farklıdır.
>
> **Ders sonrası not:** Hocanın kendi anlattığı gibi C kütüphanesi tamponludur, bu yüzden her `getc` çağrısı diske gitmez. 20 karakterlik bir satır dosyası `getc` döngüsüyle ve `fgets` döngüsüyle okundu. `strace` ile bakılınca iki programda da işletim sisteminden aynı sayıda `read` çağrısı yapıldı ve veri tek seferde 20 bayt olarak geldi (gcc ile derlendi, strace ile denendi). Yani "20 kez diske gitmek" ifadesi tamponsuz okuma için geçerlidir. Satır okumak için `fgets` kullanmak yine de doğru alışkanlıktır.

### 7.2 Konum Fonksiyonları ve Tampon Boşaltma

Hoca slayttaki bir tabloda bazı fonksiyonları gösterdi. İkisi dosya konum göstergesinin yerini ayarlıyor, biri de akışın boşaltılmasıyla (flush) ilgili. Bunlara yeri geldikçe döneceğini söyledi. Bellek kısıtınız yoksa veriyi girdi akışından okuyup bellekte tutar, sonra çıktı akışına yazabilirsiniz. Ama bazen tamponlamak gerekir ve o zaman flush kullanılır.

### 7.3 Dosyayı Kapatmak

İşimiz bitince dosyayı kapatırız. Kapatmazsak ne olur? "Program evde düzgün çalışıyordu, sonra nedense çalışmıyor" gibi durumlar çıkabilir. Dosya kapanmadıysa işletim sistemi o dosyayı hâlâ kullanımda görür. En yaygın sorun şudur: Yazmak için gönderdiğiniz veri, dosya kapanmadığı için henüz dosyaya yansıtılmamış olabilir. Sonra "bu veriyi yazmıştım, dosyada yok" dersiniz.

> **Hocanın vurgusu:** `malloc` yapınca hemen altına `free`'yi yazdığımız gibi, `fopen` yapınca hemen altına `fclose`'u yazıyoruz.
>
> **Ders sonrası not:** Denemede `fputs` ile yazılan veri, `fclose` çağrılmadan önce dosyada görünmedi (boyut 0). Program `return 0` ile normal bitince veri dosyaya geçti, `abort` ile sonlanınca geçmedi ve dosya boş kaldı (gcc ile denendi). Yani kayıp riski, program açıkken ya da anormal sonlanmada ortaya çıkar. `fclose` yazmak her durumda doğrudur.

## 8. Dosya Nasıl Okunur, Nasıl Yazılır?

Hocanın bu dersteki en önemli mesajı burasıdır. Karakter, satır, blok, bit: Hangi birimle okuyacağız? Sekiz fonksiyondan hangisini kullanacağız?

**Soru:** (Hoca sordu) Bir dosya nasıl okunur? **Cevap:** Bir öğrenci blok blok olabileceğini söyledi. Hoca bunu bir örnekle sınadı: Şu dosyayı `int main` görene kadar okuyup bütün karakterleri kaydedeceksiniz. Blok demek, bulunduğunuz yerden belli bir uzunluk almak demektir. Fakat bloğun uzunluğunu önceden bilemezsiniz, o hâlde blok işe yaramaz. Satır satır okumak da tam çözüm değildir, çünkü `int main` bir satırın ortasında da olabilir. Örneğin `typedef` satırının

yanına yazılmış olabilir. En garantisi karakter karakter okumaktır, ama biraz önce dediğimiz gibi bu da çok iş yüküdür.

Cevabın kendisi şudur:

> **Hocanın vurgusu:** Bir dosya nasıl okunur? O dosya nasıl yazıldıysa öyle okunur. (Hoca "not edin" dedi.)

Yani dosya hangi mantıkla oluşturulduysa o mantıkla okunur. Az önceki CSV dosyası virgülle ayrılmış satırlar olarak yazıldıysa şöyle okunabilir: Satırı oku, bu bir string'dir. Bu string'i virgüllere göre ayrıştır (parse). İlk parça `id`, ikincisi `ad`, üçüncüsü `soyad`'dır. Başka bir yol da virgül görene kadar okumaktır. Bu örnekte iki virgül gördüyseniz bir satırın alanlarına gelmişsiniz demektir.

Okumak için önce yazılmış olması gerekir. O zaman ikinci soru gelir.

**Soru:** (Hoca sordu) Bir dosya nasıl yazılır? **Cevap:** Nasıl istiyorsanız öyle yazılır, bir kriteri yoktur. Ama kolay okunabilir ve yazılabilir olacak şekilde düşünmek, tasarlamak gerekir. Örneğin verilerin içinde virgül olmayacağını varsayıp ayraç olarak virgül kullanabilirsiniz. Verilerde virgül olacağını düşünüyorsanız noktalı virgül seçmek daha iyi olur. O zaman dosya noktalı virgüllere göre oluşturulmuş olur.

> **Hocanın vurgusu:** Dosyayı nasıl istiyorsanız öyle yazarsınız, ama bunu mühendisliğini yaparak istemeniz lazım. Gelişigüzel tasarım ileride başınıza iş açar. Bu mantık, fonksiyonları anlatmaktan çok daha önemli bir ayrıntıdır.

## 9. Dosya Biçimleri: Başlık, Veri, Son Ek

Metin dosyalarının mantığı buydu, ikili dosyalarda daha karışık mantıklar vardır. Hoca bunların örneklerini sonraki derslerde göstereceğini söyledi. Genel dosya biçimi şöyledir:

- **Başlık (header):** Verinin nasıl düzenlendiğine dair bilgi verir. Bu, bağlı listedeki `head` değildir. İkili dosyalarda çok, metin dosyalarında az görülür. Örneğin "bu bir görüntü dosyasıdır" ya da "bu PDF şu kadar karakter içerir" diyebilir.
- **Veri (data):** Asıl içerik.
- **Son ek (footer):** Bazı durumlarda bulunur, her zaman olmaz.

### 9.1 PGM ve PPM

Hoca internette bu biçimlerin açıklamalarını ve örneklerini gösterdi. PGM gri seviyeli bir görüntü biçimidir. Gösterdiği dosyanın ilk satırları görüntünün türünü, boyutunu ve değer aralığını veriyordu: 24 sütun, 7 satır ve 0 ile 15 arasında değişen değerler. Hoca bu dosyaların IrfanView gibi bir programla açılabileceğini söyledi. Windows'taki görüntüleyici bunları açmadı. Program başlıktaki bilgileri kullanarak dosyayı görüntüye çevirir.

PPM aynı biçimin renkli görüntü hâlidir. Örnekte başlık `P3` ile başlıyor, görüntü 4'e 4'lük ve değerler 15'e kadar gidiyordu. Hoca başlığın nerede bittiğini gösterdi: O kısım başlık, altı veri, bu dosyada son ek yok.

> **Ders sonrası not:** Hoca PGM'deki türü "iki kanallı görüntü", PPM'deki `P3`'ü "3 boyutlu" diye okudu. Bu harf ve rakamlar büyük olasılıkla kanal sayısı değil, biçimin sihirli sayısıdır (magic number): `P2` metin tabanlı gri seviyeli PGM'yi, `P3` metin tabanlı renkli PPM'yi belirtir.

### 9.2 BMP

BMP biçiminin ayrıntıları çok daha kapsamlıdır. Başlıkta önce bir imza vardır. Sonra dosya boyutu gelir. Boyut lazımdır, çünkü resme çift tıkladığınızda açılan programın bellekte ne kadar yer ayıracağını bilmesi gerekir. Daha sonra henüz kullanılmayan ayrılmış (reserved) alanlar, bir dosya ofset değeri, görüntünün yatay ve düşey piksel sayısı, piksel başına bit sayısı ve metre başına piksel gibi bilgiler bulunur. Ardından görüntünün verisi gelir.

Dosyanın temeline inerseniz içinde yalnız 0 ve 1'lerden oluşan bir dizi vardır. Biçim, bu diziye anlam kazandıran anlaşmadır. Hocanın anlatımıyla "ilk şu kadar bit dosyanın imzasına karşılık

gelsin, sonraki şu kadar bit dosyanın boyutuna, sonraki alanlar da şunlara karşılık gelsin" denir ve bu dosyanın sonuna kadar böyle gider.

> **Ders sonrası not:** Hoca burada "ilk 3 bit", "sonraki 5 bit", "sonraki 8 bit" dedi. Bunlar temsilî sayılardır. Gerçek BMP'de alanlar büyük olasılıkla bayt ölçeğindedir (imza 2 bayt, dosya boyutu 4 bayt gibi).

### 9.3 Biçimi Geleceği Düşünerek Tasarlamak: IPv4 ve IPv6

Hoca, biçim tasarımının neden önemli olduğunu ağ protokolüyle anlattı. IP de kendine göre bir veri biçimidir ve hocanın benzetmesiyle bilgisayardan giden gelen veri bir dosya biçimindedir.

Evinizdeki `192.168.10.8` gibi bir adresi düşünün. Her parça 8 bittir, toplamda 32 bit eder. Yani teknik olarak $2^{32}$ cihaz adreslenebilir. Bu çok büyük bir sayı değildir. Hoca kaba hesap için $2^{30} = 8^{10}$ yazdı (altyazıda böyle geçiyor). Dünya nüfusu milyarları buldu, herkesin evinde birkaç cihaz var. Sayılar şöyledir (python ile hesaplandı):

- Verilen: IPv4 32 bit, IPv6 128 bit.
- $2^{32} = 4\,294\,967\,296$ adres.
- $2^{30} = 8^{10} = 1\,073\,741\,824$.
- IPv6'nın IPv4'e oranı: $2^{128-32} = 2^{96} = 79\,228\,162\,514\,264\,337\,593\,543\,950\,336$ kat.

**Soru:** (Hoca sordu) IPv6 çok daha fazla adres veriyor, peki neden hemen geçemiyoruz? **Cevap:** Bir öğrenci önceden atanmış çok fazla şey olduğunu ve hepsini değiştirmenin zor olacağını söyledi. Hoca bunu açtı: Telefonlarımız, bilgisayarlarımız, işletim sistemlerimiz, modemler, switch'ler, router'lar, kısacası bütün internet altyapısı IPv4 biçimine göre programlandı. Şu anda hocanın sesinin bize ulaşması bile bu protokolle oluyor ve veriler bir algoritmaya göre açılıp anlamlandırılıyor. Fiber optik, normal kablo ve telefon ağları iç içe. Bütün bunları bir anda yeni biçime almak kolay değildir. IPv6 çok daha esnektir, ama hemen geçmek şu an için mümkün değildir.

### 9.4 Biçim Üzerine Benzetmeler

Hoca, projede size bir dosya biçimi verileceğini ve ona uymanız gerektiğini söyledi. Bunu iki benzetmeyle pekiştirdi:

- **Sayısal analizde denklem biçimi:** Ödevde logaritmik bir denklemin belli bir yazımla girilmesi istenir: Örneğin 10 tabanında logaritma için alt tireli bir notasyon ve $x^2 + 5x$ yerine `x*x + 5x` gibi. Matematikte böyle bir yazım yoktur, ama "benim biçimim bu, bunu kullanacaksın" denir.
- **Protein sentezi:** Başlatıcı kodon gelir, sonra veriler üçer üçer alınır, her üçlü bir amino aside karşılık gelir ve bu durdurucu kodona kadar sürer. Biyoinformatikte de aynı biçimle çalışılır. Bağlı listede de benzerdir: `head` ile başla, `NULL` görene kadar ilerle.

  > **Hocanın vurgusu:** Dosya işlemleri sığ düşünmeyle değil, geniş düşünmeyle olur. Bu mantığı anlarsanız fonksiyonları kullanmak zor değildir.

## 10. Dosya Açma Kodu

Hoca koda geçti. Ana konu sıralı erişimli dosyalardı, rastgele erişimi (random access) sonraya bıraktı. İlk adımda `stdio.h` eklenir, çünkü `FILE` yapısının bilgileri oradadır. Sonra bir dosya işaretçisi oluşturulur ve dosya açılır. Dosya adı yalnız `deneme.txt` ise, program nerede kayıtlıysa dosya orada aranır. Hoca, programın olduğu klasöre `deneme.txt` oluşturup içine bir metin yazdı.

Dosya açma başarısız olabilir, bu yüzden sonucu kontrol etmek gerekir:

```c
#include <stdio.h>

int main(void) {
    FILE *fp;

    if ((fp = fopen("deneme.txt", "r")) == NULL) {
```

```
        printf("Dosya açılamadı\n");
        fprintf(stderr, "Dosya açma hatası\n");
    } else {
        printf("Dosya açıldı\n");
        fclose(fp);
        printf("Dosya kapatıldı\n");
    }

    return 0;
}
```

Programı çalıştırdı ve "Dosya açıldı, dosya kapatıldı" çıktısını gördü. Dosya adını bilerek yanlış yazınca (`denem.txt`) iki hata mesajı da geldi. `printf` ekrana yazar. `fprintf`'e ise nereye yazacağını söyleriz, burada `stderr`'e gönderdik.

> **Ders sonrası not:** Hoca, `if` içinde hem atama hem karşılaştırma yapabilmemizin nedenini "= operatörünün =='ye üstünlüğü var" diye açıkladı. Tersidir: == operatörünün önceliği ='den yüksektir. Atamanın önce yapılmasını sağlayan şey parantezdir. Parantez yazılmayan `if (fp = fopen("deneme.txt", "r") == NULL)` satırı, karşılaştırmanın sonucunu `fp`'ye atamaya çalıştığı için gcc 15.2'de hata verdi (`-Wint-conversion`, gcc ile denendi).

### 10.1 stdout ile stderr Sırası

Hoca `printf` ile birkaç "program içi" satır daha yazdırıp sıralamaya baktı. `printf`'lerin hepsi sırayla görünür. `stderr`'e gönderilen şey ise farklı bir kanaldan gelir ve hocaya göre yerine göre en sonda görünebilir. Hocanın ekranında sıra bozulmadı. Java gibi dillerde hatanın kırmızı renkte, farklı sırada ve yığın (stack) mantığıyla gösterilebildiğini söyledi.

> **Ders sonrası not:** `stderr`'in en sonda görünmesi kesin değildir. Çıktı boruya yönlendirilince (`./a.out 2>&1 | cat`) `stderr` mesajı `printf` satırlarından önce göründü, çünkü `stdout` tamponda bekledi (gcc ile denendi). Terminalde `stdout` genelde satır satır boşaltıldığı için sıra büyük olasılıkla yazıldığı gibi görünür.

### 10.2 Dosya Yolu ve İzinler

Dosya, programın çalıştığı yerde olmayabilir. Hoca `C:` kök dizinindeki bir dosyayı açmayı denedi: `"C:\\deneme.txt"`. Ters bölü işaretinin çift yazılması gerekir. Dosya orada yokken "açılamadı" hatası alındı. Hoca dosyayı oraya kendi eliyle kaydetti ve bu kez açıldı. Ekranda da gösterdiği gibi işletim sistemi o klasörde doğrudan dosya oluşturmasına izin vermedi. Dosya var olsa bile, güvenlikli bir klasörde işletim sistemi açmaya izin vermeyebilir.

> **Ders sonrası not:** Hoca çift yazma nedenini "iki tırnak arasında escape karakter kabul edilmediği için" diye anlattı. Doğrusu, string içinde `\` kaçış karakterinin başlangıcıdır. Bu yüzden kendisi `\\` diye yazılır. `"C:\deneme.txt"` satırı gcc'de `unknown escape sequence` uyarısı verdi (gcc ile denendi).

## 11. openFile Fonksiyonu

Hoca açma işini bir fonksiyona taşıdı. Fonksiyon dosya adını ve kipi alır, dosyayı açar ve işaretçiyi döndürür. Dönüş türü bu yüzden `FILE*`'tır. Adı ve kipi de kullanıcıdan alacak şekilde `main`'i değiştirdi:

```
#include <stdio.h>

FILE *openFile(char *fName, char *mode) {
    FILE *fp;

    if ((fp = fopen(fName, mode)) == NULL) {
        fprintf(stderr, "Dosya açma hatası\n");
    }
    return fp;
```

```
}

int main(void) {
    FILE *fp;
    char name[20], mode[3];

    printf("Sırayla dosya adı ve modu girin: ");
    scanf("%s", name);
    scanf("%s", mode);

    fp = openFile(name, mode);
    if (fp != NULL) { /* ara adım eklendi */
        fclose(fp);
    }

    return 0;
}
```

Hoca `deneme.txt` adını ve `w` kipini girdi. Program dosyayı açıp kapattı. Sonra dosyaya baktığında, az önce içine veri yazdığı dosyanın boş olduğunu gösterdi: Yazma kipinde açtığı için eski değerler silinmişti.

Derlerken çıkan bir uyarıyı hoca "cast yapmadığın için" diye yorumladı ve `fp == NULL` karşılaştırmasını açıkça yazarak giderilebileceğini söyledi. Bunun çok ciddi bir uyarı olmadığını ekledi.

> **Ders sonrası not:** Dosya açılamazsa fonksiyon `NULL` döndürür. `NULL` için `fclose` çağrılmaması gerektiğinden koddaki kontrol nota eklendi. `mode[3]` iki karakterli kipleri alır. `r+b` için en az $3+1=4$ karakter gerekir (python ile hesaplandı). Kod `gcc -Wall` ile uyarısız derlendi ve `w` girilince dosya 0 bayta indi (gcc ile denendi).

## 12. Dosya Kopyalama Örneği

Son örnekte amaç bir dosyanın içeriğini alıp başka bir dosyaya kaydetmektir. Hocaya göre dosyanın ne olduğu önemli değildir: MP3 de olabilir, C dosyası da. Bir dosya okunacak, öbürü yazılacaktır. O hâlde iki dosya işaretçisi gerekir (`fp`, `fp2`). Hoca kodu basit tutmak için `if` kontrollerini yazmadı ve metin kipinde bıraktı.

> **Ders sonrası not:** Metin kipinde kopyalama, Windows'ta büyük olasılıkla satır sonlarını dönüştürür ve ikili dosyalarda veriyi bozabilir. Herhangi bir dosyayı (MP3, PNG) kopyalarken `rb` ve `wb` kullanmak gerekir. Hocanın "binary olursa sıfır ve birlere çevirir" sözü de doğru değildir: `rb` ve `wb` kipi veriyi 0-1 metnine çevirmez, baytları aynen okur (3 baytlık dosya `rb` ile okunup baytları korunarak yazılan denemede Linux üzerinde gcc ile denendi).

### 12.1 İlk Deneme: feof ile Döngü

Hoca önce şu fikirle başladı: Dosyanın sonuna gelene kadar dön. Her adımda kaynaktan bir karakter al, hedefe yaz. Hedef dosya `w` kipiyle açıldığı için yoksa oluşur.

İlk yazdığı döngü koşuluyla program hemen bitti ve hedef dosya boş kaldı.

**Soru:** While'ın içi yanlış olabilir mi? **Cevap:** Hoca kontrol etti ve evet dedi. Koşulda "sıfır olana kadar" karşılaştırması yazılmamıştı. `feof` dosya sonuna gelinmediyse sıfır döndürür, yani `while (feof(fp))` hiç çalışmaz. Doğrusu `while (feof(fp) == 0)` yazmaktır.

> **Ders sonrası not:** Hoca önce "dosyanın sonuna geldiysen sıfır değerini elde edersin" dedi. Tersidir: `feof` dosya sonuna ulaşılmadıysa sıfır, ulaşıldıysa sıfırdan farklı döndürür. Hoca da bunu derste düzeltti. gcc denemesinde okuma öncesi `feof` 0 verdi, `while (feof(fp))` ile 0 bayt kopyalandı.

Sonra başka bir sorun çıktı: Program döngüde dönüp duruyor, dosya kopyalanmıyordu. Hoca sırasıyla şunları denedi:

1. Kipi ikili yapmayı denedi, sonra geri aldı.
2. Derlemede “permission denied” hatası görünce, arka planda hâlâ çalışan önceki programın işi engellemiş olabileceğini söyledi.
3. `c` adlı bir değişkene okuyup karakteri `%c` ile ekrana yazdırdı. Değişkenin `int` olmasında sorun olmadığını söyledi.
4. Kesme noktası (breakpoint) koyup hata ayıklayıcıyla (debug) `fp`'ye baktı. `fp`'nin 0 olduğunu, yani dosyanın açılamadığını gördü. `r+` kipini de denedi.
5. Açık programları kapatıp ortamı yeniden açtı.
6. Yeniden derleyip `c` değişkenini izledi (watch). Bu kez dosya açıldı, ilk karakter olarak 77 okundu ve kopyalama tamamlandı.

Dosyada `Merhaba` yazıyordu. `M` karakterinin ASCII kodu 77'dir (python ile hesaplandı).

**Soru:** İlk ifadeyi paranteze almamız gerekmez mi? **Cevap:** Hoca, ifadeleri ayrı yazmanın farklı bir şey değiştirmediğini, öyle bir sorun olsa derleyicinin söyleyeceğini belirtti. Asıl nedenin arka plandaki bir programın dosyayı engellemesi olduğunu düşündüğünü söyledi. Bu nedeni hoca da “muhtemelen” diye ifade etti.

### 12.2 Çalışan Kod

```c
#include <stdio.h>

FILE *openFile(char *fName, char *mode) {
    FILE *fp;

    if ((fp = fopen(fName, mode)) == NULL) {
        fprintf(stderr, "Dosya açma hatası\n");
    }
    return fp;
}

int main(void) {
    FILE *fp, *fp2;
    int c;

    fp = openFile("deneme.txt", "r");
    fp2 = openFile("deneme2.txt", "w");
    if (fp == NULL || fp2 == NULL) { /* ara adım eklendi */
        return 1;
    }

    while ((c = getc(fp)) != EOF) {
        putc(c, fp2);
    }

    fclose(fp);
    fclose(fp2);
    return 0;
}
```

Hoca, karakter karakter okumanın basit bir yöntem olduğunu ve çok tavsiye etmediğini de hatırlattı.

> **Ders sonrası not:** Bu kod `gcc -Wall` ile uyarısız derlendi ve 12 baytlık `Merhaba YTU\n` dosyasını 12 bayt olarak kopyaladı (`cmp` ile aynı çıktı, gcc ile denendi). `while (feof(fp) == 0) putc(getc(fp), fp2);` döngüsü ise 13 bayt yazdı ve sonda fazladan `ff` baytı oluştu. Çünkü `feof`, ancak okuma dosya sonunu geçtikten sonra doğru olur. Son `getc`'nin döndürdüğü `EOF` de hedefe yazılır.

> **Ders sonrası not:** Hoca “integer almakta sıkıntı yok” dedi. `c`'nin `int` olması aslında gereklidir. `char c` ile `A`, `0xFF`, `B` baytlarından oluşan 3 baytlık dosyayı kopyalayınca döngü `0xFF`'te durdu ve 1 bayt kopyalandı (x86-64 üzerinde gcc ile denendi).

## 13. Dönem Projesinin Teknik Yapısı

Hoca dersin sonunda projeyi tanıttı. Bu bölümde yalnız teknik gereksinimler yer alır. Proje, kütüphane otomasyonuna benzer bir veri tabanı uygulamasıdır. Hoca, OBS'nin küçük ölçekli hâli gibi düşünebileceğimizi söyledi. Her veri hem CSV dosyasında kalıcı olarak saklanır hem de programda yapılar (struct) ve bağlı listeler ile tutulur. Yapılar ve dosyaların birlikte kullanımının sonraki derste ele alınacağını belirtti.

Beş CSV dosyası ve bunlara denk gelen yapılar vardır:

- **Yazarlar:** Ad, soyad ve her yazar eklemede otomatik verilen yazar ID'si.
- **Öğrenciler:** Ad, soyad, kütüphane puanı ve her öğrenci için benzersiz öğrenci numarası.
- **Kitaplar:** Kitap adı, ISBN numarası ve adet bilgisi. Örnek kayıt: `Java Programlama,<ISBN>,<adet>`. Bir kitabın birden fazla örneği olabileceği için her nüshaya, ISBN'in kaçıncı nüshası olduğunu gösteren bir etiket verilir (`ISBN_1` gibi).
- **Kitap-yazar eşleştirmesi:** ISBN ile yazar ID'sini eşleştirir. Bir kitabı birden fazla yazar yazabilir, bir yazar da birden fazla kitap yazabilir. Yazar ID'si, yazarlar dosyasındaki yazarın adına ve soyadına bağlanır. Yazar silinirse ya da henüz atanmadıysa ID alanı $-1$ olur.
- **Ödünç alma işlemleri:** Kitabın nüshası, öğrenci, tarih ve işlem türü. İşlem türü ödünç alma için 0, teslim için 1 ile kodlanır.

Uygulamanın kütüphane kuralı şudur: Kitap ödünç alındıktan sonra 15 günden geç teslim edilirse öğrencinin kütüphane puanı güncellenir ve öğrenci cezalı öğrenci listesine girer. Hocanın dosyadaki örnek kayıtla yaptığı hesap şöyledir:

- Verilen: Ödünç alma günü 12, teslim günü 28.
- İşlem: $28 - 12 = 16$ gün (python ile hesaplandı).
- Sonuç: $16 > 15$ olduğundan bu öğrencinin puanı güncellenir (son adım eklendi).

Programda istenen işlemler:

- **Öğrenci:** Yeni öğrenci ekleme, silme, güncelleme, öğrenci bilgisini görüntüleme, cezalı öğrenci listesi, tüm öğrencilerin listelenmesi.
- **Kitap:** Yeni kitap ekleme, kitap bilgilerini görüntüleme.
- **Yazar:** Yazar ekleme, yazarlarla kitapları ilişkilendirme.

Ekleme, silme, güncelleme ve okuma (CRUD) işlemlerinin çoğu ID üzerinden yapılır.

## Dersin Toparlaması

Ders iki ana konudan oluştu. Birincisi void işaretçiydi: Her türün adresini tutan bu işaretçi, kullanılmadan önce asıl türüne çevrilir ve adres aritmetiği bayt bayt elle kurulur. `malloc`'un her türe bellek ayırabilmesi de bu sayede mümkündür. İkincisi dosyalara giriş oldu: Kalıcılık için veriyi diske yazarız, bu iş işletim sisteminin gözetiminde akışlar üzerinden yürür. Metin ve ikili dosya ayrımını, `fopen` kiplerini, dosya işaretçisi ile konum göstergesi ayrımını, dosya okuma birimlerini ve `fclose` gerekliliğini gördük.

Dersin asıl mesajı fonksiyonlardan çok tasarımdır: Dosya nasıl yazıldıysa öyle okunur ve biçim, geleceği düşünerek mühendislikle tasarlanır. Yazılan dosya kopyalama örneği karakter karakter çalışır. Hoca sonraki derste dosya işlemlerine kalınan yerden devam edeceğini, yapılar ile dosyaların birlikte kullanımını ve BMP gibi ikili biçim örneklerini göstereceğini söyledi.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.